

166 CSE 4802 — Compiler Design Lab

Section 1B | Assignment 1

Lexical Analyzer for a Language with Token Classification and Statistics

1. Experiment Title

Lexical Analyzer for a Language with Token Classification and Statistics

2. Objectives

After completing this lab, students are able to:

- Design a lexical analyzer for a restricted language.
- Apply regular expressions to classify tokens.
- Distinguish between different categories and subcategories of operators.
- Recognize both integer and floating-point constants.
- Maintain a symbol table for identifiers.
- Generate token statistics and frequency reports.

3. Background Theory

3.1 Lexical Analysis in Compiler Design

Lexical analysis is the first phase of a compiler. It converts a raw sequence of characters from the source code into a structured sequence of tokens. These tokens are then consumed by later phases of the compiler, most notably the parser.

3.2 Key Concepts

Concept	Definition
Token	A syntactic category (e.g., IDENTIFIER, KEYWORD)
Lexeme	The actual string matched in the source code
Pattern	The rule used to identify a token (via regular expressions)
Transition Diagram	A state machine diagram used to identify and accept tokens

3.3 Lexical Analysis Workflow

The lexical analyzer follows these steps:

1. Read the source code file line by line.
2. Group individual characters into candidate lexemes.
3. Match each lexeme against known token patterns.
4. Classify and emit the recognized token.
5. Produce reports and detect lexical errors.

4. Problem Statement

Design a lexical analyzer that:

- Reads a program written in a subset language.
- Identifies and classifies tokens using regular expressions (manual implementation — no Lex/Flex).
- Produces a token stream, token statistics, and a symbol table.
- Reports lexical errors (invalid identifiers, unknown symbols).

5. Functional Requirements

The program recognizes the following token categories:

Token Category	Examples / Description
Keywords	int, float, char, if, else, while, for, return, ...
Identifiers	User-defined names: letters, digits, underscore
Integer Const	Whole numbers, e.g. 10, 0, 42
Float Const	Decimal numbers, e.g. 3.14, 2.718
Char Const	Single character in quotes, e.g. 'x'
String Const	Text in double quotes, e.g. "hello"
Arithmetic Op	+ - * / %
Relational Op	== != < > <= >=
Logical Op	&& !
Bitwise Op	& ^ ~ << >>
Assignment Op	= += -= *= /= %= &= = ^= <<= >>=
Incr/Decr Op	++ --
Delimiters	(){}[]; , .

Additional rules:

- Whitespace, tabs, and newlines are ignored (except for tracking line numbers).
- Keywords take priority over identifiers.
- Longest match rule is enforced.
- Invalid tokens (e.g. 9abc) are reported as errors.

6. Algorithm (Guideline)

6. Read the input file line by line.
7. Initialize token counters and symbol table.
8. For each line, scan characters sequentially to form lexemes.
9. Apply matching rules in priority order:
 - Check multi-character operators first (longest match).
 - Then single-character operators.

- Then keywords and identifiers.
 - Then constants (float before integer).
 - Then delimiters.
10. Classify the token and store the result.
 11. Update counters and the symbol table.
 12. Report errors if no pattern matches.
 13. After all lines: print token stream, statistics, and symbol table.

7. Design Constraints

The following tools are NOT used:

- Lex / Flex
- Yacc / Bison

Implementation language: Python 3

Standard string processing and regular expressions (re module) are used.

8. Source Code (Python)

lexer.py

```
"""
CSE 4802 Compiler Design Lab - Section 1B: Assignment 1
Lexical Analyzer with Token Classification and Statistics
"""

import re, sys
from collections import defaultdict

KEYWORDS = {
    'int', 'float', 'char', 'double', 'long', 'short', 'unsigned', 'signed',
    'void', 'if', 'else', 'while', 'for', 'do', 'return', 'break', 'continue',
    'switch', 'case', 'default', 'struct', 'union', 'typedef', 'enum',
    'const', 'static', 'extern', 'sizeof', 'printf', 'scanf', 'include', 'main'
}

MULTI_CHAR_OPERATORS = [
    ('++', 'INCR_DECR_OP'), ('--', 'INCR_DECR_OP'),
    ('+=', 'ASSIGNMENT_OP'), ('-=', 'ASSIGNMENT_OP'), ('*=', 'ASSIGNMENT_OP'),
    ('/=', 'ASSIGNMENT_OP'), ('%=', 'ASSIGNMENT_OP'), ('&=', 'ASSIGNMENT_OP'),
    ('|=', 'ASSIGNMENT_OP'), ('^=', 'ASSIGNMENT_OP'),
    ('<=', 'ASSIGNMENT_OP'), ('>=', 'ASSIGNMENT_OP'),
    ('<<', 'BITWISE_OP'), ('>>', 'BITWISE_OP'),
    ('==', 'RELATIONAL_OP'), ('!=', 'RELATIONAL_OP'),
    ('<', 'RELATIONAL_OP'), ('>', 'RELATIONAL_OP'),
    ('&&', 'LOGICAL_OP'), ('||', 'LOGICAL_OP'),
]

SINGLE_CHAR_OPERATORS = {
    '+': 'ARITHMETIC_OP', '-': 'ARITHMETIC_OP', '*': 'ARITHMETIC_OP',
    '/': 'ARITHMETIC_OP', '%': 'ARITHMETIC_OP',
    '<': 'RELATIONAL_OP', '>': 'RELATIONAL_OP', '!': 'LOGICAL_OP',
    '&': 'BITWISE_OP', '|': 'BITWISE_OP', '^': 'BITWISE_OP', '~': 'BITWISE_OP',
    '=': 'ASSIGNMENT_OP',
}

DELIMITERS = set('(){};,;[.]')

RE_FLOAT = re.compile(r'^\d+\.\d+([eE][+-]?\d+)?')
RE_INT = re.compile(r'^\d+')
RE_ID = re.compile(r'^[a-zA-Z_][a-zA-Z0-9_]*')
RE_CHAR_LIT = re.compile(r"'^(\.|\.[^\"])*'")
RE_STR_LIT = re.compile(r"'^\"(\.|\.[^\"])*\"")
RE_INVALID_ID = re.compile(r'^\d+[a-zA-Z_]\w*')

class Lexer:
    def __init__(self):
        self.tokens = []
        self.errors = []
        self.symbol_table = {}
```

```
self.stats = defaultdict(int)

def tokenize(self, source_code):
    for line_no, line in enumerate(source_code.splitlines(), 1):
        self._scan_line(line, line_no)
    return self.tokens

def _scan_line(self, line, line_no):
    i, n = 0, len(line)
    while i < n:
        if line[i] in '\t\r': i += 1; continue
        if line[i:i+2] == '//': break
        remaining = line[i:]
        # String literal
        m = RE_STR_LIT.match(remaining)
        if m: self._add_token('STRING_CONST', m.group(), line_no); i +=
len(m.group()); continue
        # Char literal
        m = RE_CHAR_LIT.match(remaining)
        if m: self._add_token('CHAR_CONST', m.group(), line_no); i +=
len(m.group()); continue
        # Multi-char operators (longest match)
        matched = False
        for op, op_type in sorted(MULTI_CHAR_OPERATORS, key=lambda x: -
len(x[0])):
            if remaining.startswith(op):
                self._add_token(op_type, op, line_no); i += len(op); matched
= True; break
        if matched: continue
        # Single-char operator
        if line[i] in SINGLE_CHAR_OPERATORS:
            self._add_token(SINGLE_CHAR_OPERATORS[line[i]], line[i],
line_no); i += 1; continue
        # Delimiter
        if line[i] in DELIMITERS:
            self._add_token('DELIMITER', line[i], line_no); i += 1; continue
        # Invalid identifier
        m = RE_INVALID_ID.match(remaining)
        if m: self.errors.append(("Invalid identifier: '{m.group()}'",
line_no)); i += len(m.group()); continue
        # Float constant
        m = RE_FLOAT.match(remaining)
        if m: self._add_token('FLOAT_CONST', m.group(), line_no); i +=
len(m.group()); continue
        # Integer constant
        m = RE_INT.match(remaining)
        if m: self._add_token('INTEGER_CONST', m.group(), line_no); i +=
len(m.group()); continue
        # Identifier or keyword
        m = RE_ID.match(remaining)
        if m:
            lexeme = m.group()
```

```
        if lexeme in KEYWORDS: self._add_token('KEYWORD', lexeme,
line_no)
        else:
            self._add_token('IDENTIFIER', lexeme, line_no)
            if lexeme not in self.symbol_table:
                self.symbol_table[lexeme] = {'count': 0, 'first_line':
line_no}
            self.symbol_table[lexeme]['count'] += 1
            i += len(lexeme); continue
        # Unknown
        self.errors.append((f"Unknown symbol: '{line[i]}'", line_no)); i +=
1

def _add_token(self, token_type, lexeme, line_no):
    self.tokens.append((token_type, lexeme, line_no))
    self.stats[token_type] += 1
```

9. Sample Input

```
int a = 10;
float b = 3.14;
if (a >= b && b != 0) {
    a++;
}
int sum = a + b;
char c = 'x';
int x = a << 2;
int y = a & b;
int z = a | b;
float result = (a * b) / (a - b);
```

10. Expected Output

10.1 Token Stream

Token Type	Lexeme	Line No.
KEYWORD	int	1
IDENTIFIER	a	1
ASSIGNMENT_OP	=	1
INTEGER_CONST	10	1
DELIMITER	;	1
KEYWORD	float	2
IDENTIFIER	b	2
ASSIGNMENT_OP	=	2
FLOAT_CONST	3.14	2
DELIMITER	;	2
KEYWORD	if	3
DELIMITER	(	3
IDENTIFIER	a	3
RELATIONAL_OP	>=	3
IDENTIFIER	b	3
LOGICAL_OP	&&	3
IDENTIFIER	b	3
RELATIONAL_OP	!=	3
INTEGER_CONST	0	3
DELIMITER	)	3
DELIMITER	{	3

IDENTIFIER	a	4
INCR_DECR_OP	++	4
DELIMITER	;	4
DELIMITER	}	5
KEYWORD	int	6
IDENTIFIER	sum	6
ASSIGNMENT_OP	=	6
IDENTIFIER	a	6
ARITHMETIC_OP	+	6
IDENTIFIER	b	6
DELIMITER	;	6
KEYWORD	char	7
IDENTIFIER	c	7
ASSIGNMENT_OP	=	7
CHAR_CONST	'x'	7
DELIMITER	;	7
KEYWORD	int	8
IDENTIFIER	x	8
ASSIGNMENT_OP	=	8
IDENTIFIER	a	8
BITWISE_OP	<<	8
INTEGER_CONST	2	8
DELIMITER	;	8
KEYWORD	int	9
IDENTIFIER	y	9
ASSIGNMENT_OP	=	9
IDENTIFIER	a	9
BITWISE_OP	&	9
IDENTIFIER	b	9
DELIMITER	;	9
KEYWORD	int	10
IDENTIFIER	z	10
ASSIGNMENT_OP	=	10
IDENTIFIER	a	10

BITWISE_OP		10
IDENTIFIER	b	10
DELIMITER	;	10
KEYWORD	float	11
IDENTIFIER	result	11
ASSIGNMENT_OP	=	11
DELIMITER	(	11
IDENTIFIER	a	11
ARITHMETIC_OP	*	11
IDENTIFIER	b	11
DELIMITER	)	11
ARITHMETIC_OP	/	11
DELIMITER	(	11
IDENTIFIER	a	11
ARITHMETIC_OP	-	11
IDENTIFIER	b	11
DELIMITER	)	11
DELIMITER	;	11

10.2 Token Statistics

Metric	Count
Total Tokens	73
Keywords	9
Identifiers	23
Constants (Total)	5
Integer Constants	3
Float Constants	1
Character Constants	1
String Constants	0
Operators (Total)	19
Arithmetic Operators	4
Relational Operators	2
Logical Operators	1
Bitwise Operators	3

Assignment Operators	8
Increment/Decrement Operators	1
Delimiters	17

10.3 Symbol Table

Identifier	Frequency (Count)	First Occurrence (Line)
a	9	1
b	8	2
c	1	7
result	1	11
sum	1	6
x	1	8
y	1	9
z	1	10

10.4 Lexical Errors

No lexical errors detected in the sample input.

If a token like '9abc' were present, the error report would read:

```
[Line X] ERROR: Invalid identifier: '9abc'
```

11. Transition Diagram for Token Recognition

The following state-transition table describes the DFA (Deterministic Finite Automaton) logic implemented in the lexer. The lexer starts at state S0 and transitions based on the current character. On reaching an accepting state, it emits the corresponding token and returns to S0.

State	Input Condition	Action / Next State
S0 (Start)	Whitespace / Tab / Newline	Skip → S0
S0	Letter or _	Accumulate → S1 (Identifier)
S1 (Identifier)	Letter, Digit, or _	Accumulate → S1
S1	Other character	Emit KEYWORD or IDENTIFIER → S0
S0	Digit	Accumulate → S2 (Integer)
S2 (Integer)	Digit	Accumulate → S2
S2	Dot (.)	Accumulate → S3 (Float)
S2	Other	Emit INTEGER_CONST → S0
S3 (Float)	Digit	Accumulate → S3
S3	Other	Emit FLOAT_CONST → S0
S0	Single quote (')	Accumulate → S4 (Char)
S4 (Char)	Any char or escape	Accumulate → S5
S5	Single quote (')	Emit CHAR_CONST → S0
S0	Double quote (")	Accumulate → S6 (String)
S6 (String)	Any char or escape	Accumulate → S6
S6	Double quote (")	Emit STRING_CONST → S0
S0	+ seen	Peek next → S7
S7	+	Emit INCR_DECR_OP (++) → S0
S7	=	Emit ASSIGNMENT_OP (+=) → S0
S7	Other	Emit ARITHMETIC_OP (+) → S0
S0	> seen	Peek next → S8
S8	>	Peek for = → BITWISE_OP (>> or >>=)
S8	=	Emit RELATIONAL_OP (>=) → S0
S8	Other	Emit RELATIONAL_OP (>) → S0
S0	< seen	Peek next → S9
S9	<	Peek for = → BITWISE_OP (<< or <<=)
S9	=	Emit RELATIONAL_OP (<=) → S0
S9	Other	Emit RELATIONAL_OP (<) → S0
S0	& seen	Peek next → S10
S10	&	Emit LOGICAL_OP (&&) → S0
S10	Other	Emit BITWISE_OP (&) → S0

S0	Delimiter () { } [] ; ,	Emit DELIMITER → S0
S0	Unrecognized	Report lexical error → S0

Key design decisions:

- Longest Match Rule: Multi-character operators (e.g. >=, >>=, &&) are checked before single-character ones.
- Keyword vs Identifier: After a full identifier is accumulated, it is checked against the keyword set.
- Float before Integer: RE_FLOAT is applied before RE_INT to correctly match 3.14 as a float.
- Invalid Identifier check: RE_INVALID_ID (e.g. 9abc) is tested before the integer rule to catch illegal tokens.

12. Submission Summary

Item	Details
Source Code	lexer.py (Python 3)
Input File	Sample input with 11 lines of C-like code
Token Stream	73 tokens classified and listed in section 10.1
Token Statistics	Full breakdown in section 10.2
Symbol Table	8 unique identifiers tracked in section 10.3
Transition Diagram	DFA state table in section 11